Gostim2: A Cross-Platform, Fixed-Schedule Stimulus Delivery System for Neuroimaging and Behavioral Experiments

Christophe Pallier

INSERM–CEA Cognitive Neuroimaging Unit, NeuroSpin, Gif-sur-Yvette, France

christophe@pallier.org

Abstract

We describe Gostim2, an open-source, cross-platform stimulus delivery system designed for behavioral and neuroimaging experiments in which the timing and content of all stimuli are fully specified in advance. The software reads a standard CSV or TSV file whose rows define stimulus onset times, durations, types, and content; no programming knowledge is required to build an experiment. Gostim2 is written in the Go programming language and uses the SDL3 graphics library for hardware-accelerated rendering. Visual stimuli (images, text, colored rectangles), audio stimuli (WAV, FLAC, MP3, OGG), video, and rapid serial presentation streams are supported. Temporal precision rests on two mechanisms: in the default VSYNC mode, stimulus onsets are aligned to the monitor's vertical blanking interval, yielding frame-accurate presentation; in Variable Refresh Rate (VRR) mode, VSYNC is disabled and a 2 ms busy-wait loop is used to hit scheduled timestamps with sub-millisecond accuracy on suitable hardware. All keypress events are recorded with timestamps in a plain-text log file. Hardware trigger output for synchronization with EEG, MEG, and fMRI recording systems is supported via the DLP-IO8-G USB device. The software ships as a single self-contained binary — requiring no runtime installation — and provides both a graphical user interface and a command-line interface suitable for automated or headless deployment. Gostim2 is released under the Apache License 2.0 and available at https://github.com/chrplr/gostim2.

Keywords: stimulus presentation, timing precision, neuroimaging, fixed schedule, fMRI, EEG,

SDL3, Go

Introduction

Precise control of stimulus timing is a core technical requirement in behavioral and neuroimaging experiments. In electroencephalography (EEG), event-related potentials are time-locked to stimulus onsets at the level of individual milliseconds; in magnetoencephalography (MEG) the same constraint applies; in functional MRI (fMRI), the hemodynamic response is slower but the accurate association of stimulus events with scanner volumes requires reliable trigger timing to within a fraction of a TR. Experiment-control software must therefore deliver stimuli at scheduled times with minimal and predictable jitter, record responses with accurate timestamps, and send synchronization pulses to recording hardware at the moment of each event.

Several software packages are widely used for this purpose. The MATLAB Psychophysics Toolbox (Brainard, 1997; Kleiner et al., 2007) and its Python port PsychoPy (Peirce et al., 2019) are general-purpose: they support real-time branching, participant-contingent feedback, adaptive procedures, and a large variety of stimulus types. Expyriment (Krause & Lindemann, 2014) offers a simpler Python API with a similar capability profile. These tools are appropriate for a wide range of paradigms but carry a corresponding complexity: experiments are written as programs, runtime environments must be installed and maintained across laboratory computers, and the garbage collectors of Python and MATLAB can introduce unpredictable pauses during timing-critical sections if experiments are not carefully constructed. E-Prime (Schneider et al., 2002) provides a graphical design environment at the cost of being proprietary and restricted to Windows. Web-based tools such as jsPsych (de Leeuw, 2015) and Gorilla (Anwyl-Irvine et al., 2021) enable large-scale online data collection but are subject to browser and OS compositor latency that makes frame-accurate timing difficult (Bridges et al., 2020).

A substantial class of neuroimaging experiments — particularly passive viewing and listening paradigms, naturalistic stimulation experiments, and many localizer tasks — does not require any of the real-time adaptability that makes general-purpose tools complex. In these paradigms, the full stimulus sequence is known before the session begins; the experiment is simply a precise playback machine. For this class of experiment, a leaner tool is preferable: one that can be operated without

programming, deployed to a new computer without installing a language runtime, and that focuses all of its computational resources on temporal precision rather than on supporting a general scripting environment.

Gostim2 is built to fill this niche. It accepts a stimulus schedule as a plain CSV or TSV file and presents each stimulus at its scheduled time. It does not support real-time branching or adaptive feedback; this constraint is a deliberate design choice that enables reliable resource pre-loading, predictable memory allocation, and a tight VSYNC-locked rendering loop. The software compiles to a single binary with no external dependencies and provides both a graphical user interface for interactive laboratory use and a command-line interface for scripted or headless operation.

Software Description

Implementation and Dependencies

Gostim2 is written in Go (version 1.25 or later) and uses the following third-party libraries:

- `github.com/Zyko0/go-sdl3` — SDL3 bindings for window management, 2D rendering, audio streaming, and event handling.
- `github.com/BurntSushi/toml` — TOML configuration file parsing.
- `github.com/funatsufumiya/go-gv-video` — decoding of `.gv` (LZ4-compressed RGBA) video sequences.
- `go.bug.st/serial` — serial-port communication for the DLP-IO8-G trigger device (no CGo required).

The SDL3 binary is not required on the target machine; it is bundled at build time through the `go-sdl3/bin` package and loaded at runtime, so the resulting binary is fully self-contained.

Fixed-Schedule Architecture

The central architectural constraint of Gostim2 is that the complete stimulus schedule is known before execution begins. At startup, the CSV/TSV file is parsed and validated; all referenced assets (images, audio files, fonts) are loaded into memory. Because no disk I/O occurs during the experiment, I/O latency cannot cause dropped frames or audio glitches. Stimulus rendering

parameters (scaled dimensions, center coordinates) are computed from the screen configuration at load time and cached, keeping per-frame CPU work minimal.

This architecture excludes paradigms that require real-time branching (e.g., providing feedback contingent on the participant's response, or adapting subsequent stimuli based on earlier responses). For paradigms where such adaptability is needed, the companion framework `goxpyriment` (a Go library for general-purpose experiment programming) is more appropriate.

Stimulus Configuration Format

An experiment is fully defined by a CSV or TSV file with four mandatory columns:

Column	Description
`onset_time`	Scheduled onset of the stimulus, in milliseconds from the start of the run
`duration`	Duration of the stimulus display or playback, in milliseconds
`type`	Stimulus category: `IMAGE`, `SOUND`, `TEXT`, `BOX`, `VIDEO`, `IMAGE_STREAM`, `TEXT_STREAM`, `SOUND_STREAM`, or `END`
`stimuli`	File path, text content, or stream specification

For stream types (`IMAGE_STREAM`, `TEXT_STREAM`, `SOUND_STREAM`), multiple items are listed in a single `stimuli` cell separated by `~` characters. Per-item durations and inter-stimulus gaps can be specified inline using `:duration:gap` suffixes, allowing complex RSVP sequences to be expressed in a single CSV row. The `END` type stops the experiment at a specified time; if omitted, the experiment concludes after the last scheduled event.

Optional columns include `trigger` (a byte value sent to the DLP-IO8-G at onset) and any number of additional metadata columns that are passed through to the event log.

A separate TOML configuration file stores session-level parameters: subject ID, screen resolution, window mode, display index, font, scale factor, background color, text color, fixation color, stimuli directory, results directory, and DLP device path. This file is created automatically by the GUI after each session and can be passed to the CLI with `-config <path>` to reproduce the same setup.

Temporal Precision

Precision is the primary technical objective of Gostim2. Two operating modes are provided, selected via command-line flags or GUI checkboxes:

VSYNC mode (default, `-vsync`). The rendering loop calls `SDL_RenderPresent`, which blocks until the monitor's vertical blanking interval. On a 60 Hz monitor this is approximately 16.67 ms per frame; on a 120 Hz monitor approximately 8.33 ms. To avoid missing a target frame, the engine uses a predictive look-ahead window (nominally half a frame duration). If the scheduled onset of the next stimulus falls within this window before the current VBLANK, the renderer prepares it immediately so that it appears on the next flip rather than the one after. This mechanism ensures frame-accurate stimulus onsets under typical operating conditions. The precision of this approach is identical in principle to that of the Psychophysics Toolbox's Screen('Flip') mechanism (Brainard, 1997; Kleiner et al., 2007).

VRR mode (`-vrr`). VSYNC is disabled and the engine replaces the blocking flip with a 2 ms busy-wait loop. The loop polls the system clock and renders at the precise scheduled timestamp without waiting for a frame boundary. On systems equipped with G-Sync or FreeSync displays — which present a frame immediately when signaled rather than at a fixed interval — this mode allows stimulus onsets to be placed at any point in time, limited only by system clock resolution and OS scheduling jitter. The busy-wait duration (2 ms) is a configurable trade-off between CPU load and temporal resolution.

In both modes, Go's garbage collector could in principle pause the timing loop. For the relatively coarse delivery loop of Gostim2 (stimuli typically on the order of hundreds of milliseconds to seconds), GC pauses of a few milliseconds are unlikely to cause missed frames. For the dense RSVP stream types, GC pressure is reduced by pre-loading all assets and avoiding allocation in the hot loop.

Stimulus Types

Gostim2 renders the following stimulus categories:

- **IMAGE**: A static image file displayed for the specified duration. Supported formats include PNG, JPEG, BMP, and any format handled by SDL3's image loader. Images are scaled according to the global scale factor and centered on screen by default.
- **TEXT**: A text string rendered in the configured font and color.

- **BOX**: A filled colored rectangle, useful for visual masks, blank intervals with non-default background color, or simple geometric stimuli.
- **SOUND**: An audio file played for its natural duration or the specified duration. Supported formats: WAV, FLAC, MP3, OGG. All audio is resampled at load time to the device's native sample rate, eliminating resampling overhead during playback.
- **VIDEO**: A video file played frame-by-frame, VSYNC-locked. The `.gv` (LZ4-compressed RGBA) format is natively supported; MPEG is also handled.
- **IMAGE_STREAM / TEXT_STREAM**: A rapid serial sequence of images or text items presented within a single CSV row, with per-item onset asynchronies specified inline. This supports RSVP and temporal masking paradigms.
- **SOUND_STREAM**: A sequence of audio clips played in rapid succession, analogous to visual streams.

A superimposed fixation cross can be configured to appear on blank intervals, always, or never, independently of the stimulus schedule.

Audio

Audio is handled by a custom software mixer implemented directly over SDL3's audio stream API. Up to 16 sounds can play simultaneously; the mixer sums their samples with saturation at 16-bit limits. All audio files are decoded and resampled to the device's native format (default: 44100 Hz, stereo, signed 16-bit) during the pre-loading phase. Because audio data is already in the device's format when playback begins, there is no per-sample conversion overhead at play time and sound onset is not delayed by decompression.

Event Logging

All keypress events — including the key code, timestamp relative to experiment start (in milliseconds), and the identity of the currently displayed stimulus — are recorded in a plain-text log file written to the results directory. The filename encodes the experiment name, subject ID, and a timestamp to prevent overwriting. Header lines record the configuration parameters used for the session, providing a self-contained record of each run.

Hardware Trigger Output

For EEG, MEG, and fMRI synchronization, Gostim2 supports the DLP-IO8-G USB device, which presents as a virtual serial port and controls eight TTL output lines. The three lower lines are mapped to stimulus categories: line 1 to IMAGE and VIDEO onsets, line 2 to SOUND onsets (with a fixed 5 ms pulse), and line 3 to TEXT and BOX onsets. The DLP device path is specified in the configuration; the engine opens the port at startup and closes it cleanly on exit. An `AutoDetect` function scans available serial ports for a DLP-IO8-G signature, returning a no-op implementation if no device is found, so that the same experiment file runs unmodified on systems without recording hardware.

User Interfaces

Both interfaces share the same underlying engine, configuration struct, and event log format:

GUI (`gostim2-gui`). A graphical SDL3 dialog presents labeled text fields for the configuration file path, subject ID, CSV file, start/end splash screens, font, results directory, and DLP device path; numeric fields for display index, font size, and scale factor; color pickers for background, text, and fixation colors; and checkbox toggles for VSYNC, VRR, autodetect resolution, fullscreen mode, skip-wait, and fixation mode. Settings are persisted to `gostim2_config.toml` after each session and pre-filled on the next run.

CLI (`gostim2`). A command-line interface accepting flags corresponding to each configuration field. The same TOML file used by the GUI can be loaded with `-config <path>`, allowing the GUI to be used for interactive setup and the CLI for automated scripted runs (e.g., from an MRI scanner room trigger script).

Display Modes

Three window modes control how Gostim2 interacts with the OS display stack:

- **Windowed** (`window_mode = 0`): a resizable window at the specified resolution. Suitable for development and testing.
- **Fullscreen desktop** (`window_mode = 1`): a borderless fullscreen window at the current desktop resolution. The OS compositor may introduce one frame of additional buffering.
- **Fullscreen exclusive** (`window_mode = 2`): requests exclusive access to the display at the

specified mode, bypassing the compositor. This mode provides the most predictable frame delivery on Windows and macOS.

On Linux, additional precision can be obtained by running the CLI directly from a TTY console (with the display manager stopped), allowing SDL3 to interface with the Direct Rendering Manager (DRM) without any compositor overhead.

Example Experiments

The repository includes three complete working experiments:

- **Jabberwocky**: An auditory language experiment presenting words and non-words from Lewis Carroll's poem with precise SOAs, accompanied by trigger pulses for EEG/MEG.
- **La Cigale et la Fourmi**: A naturalistic listening experiment presenting a spoken French poem stimulus, illustrating the use of the `SOUND` type with long continuous audio.
- **Visual Categories Localizer (demo)**: A visual category localizer adapted from Minye Zhan, presenting images from multiple categories (faces, objects, scenes, scrambled) at 150 ms per image in RSVP streams, with trigger output and a full configuration file.
- **Simple Localizer (demo)**: A simple auditory localizer adapted from Philippe Pinel, demonstrating TEXT, IMAGE, and SOUND stimulus types in a single experiment.

All examples include the stimulus files, CSV schedule, and TOML configuration needed to run them immediately after installation.

Comparison with Existing Tools

Table 1 compares Gostim2 with selected tools used for stimulus presentation in behavioral and neuroimaging research.

Table 1. Comparison of selected stimulus presentation systems.

Property	Gostim2	Expyriment	PsychoPy	E-Prime	jsPsych
Language	Go	Python	Python	Proprietary	JavaScript
Experiment specification	CSV/TSV file	Python script	Python script / Builder	GUI	JavaScript
Programming required	No	Yes	Optional	Optional	Yes
Deployment	Single binary	pip/conda env	Standalone or pip	Windows installer	Browser/Node.js
Real-time branching	No	Yes	Yes	Yes	Yes
VSYNC sync	Yes (SDL3)	Yes (Pygame)	Yes (Pyglet/PTB)	Yes	Limited (rAF)
Sub-frame timing (VRR)	Yes (busy-wait)	No	No	No	No
Custom audio mixer	Yes	No	No	No	No
Hardware triggers	DLP-IO8-G, parallel	Parallel, serial	Parallel, serial	Serial	No
EEG/MEG/fMRI suitability	High (fixed-schedule)	Medium	Medium	High	Low
License	Apache 2.0	GPL v3	GPL v3	Proprietary	MIT
Reference	This paper	Krause & Lindemann, 2014	Peirce et al., 2019	Schneider et al., 2002	de Leeuw, 2015

The most distinctive characteristic of Gostim2 relative to general-purpose tools is its fixed-schedule constraint. By forgoing real-time adaptability, the engine can pre-load all assets, avoid dynamic memory allocation in the hot loop, and maintain a tight VSYNC-locked rendering cycle. Researchers who need adaptive procedures, participant feedback, or trial-by-trial parameter variation should use a general-purpose tool. Researchers running fixed-sequence paradigms — the majority of fMRI localizers, MEG passive listening tasks, and many RSVP experiments — can describe their

experiment as a spreadsheet and rely on Gostim2 for delivery without writing any code.

The absence of a Python runtime dependency is a practical advantage in multi-user laboratory environments: Gostim2 is distributed as a pre-built binary for Windows (x86_64 and ARM64), macOS (Intel and Apple Silicon), and Linux (x86_64 AppImage and Debian package), and requires no installation beyond placing the binary on the target machine.

Limitations and Future Directions

Fixed-schedule constraint. The design intentionally excludes real-time feedback and adaptive stimulus selection. This is a feature for the target use case but a hard limitation for paradigms requiring contingent responses.

No formal timing validation study. The temporal precision mechanisms described — VSYNC synchronization and VRR busy-wait — are well-established approaches (Brainard, 1997; Kleiner et al., 2007), but no published photodiode or oscilloscope measurements specific to Gostim2 on particular hardware configurations currently exist. Such measurements are a priority for future work.

Trigger mapping is fixed. The current DLP-IO8-G trigger assignment (line 1: images/video; line 2: sounds; line 3: text/boxes) is hardcoded. Future versions should allow per-stimulus trigger values to be specified directly in the CSV.

Audio mixing is software-only. The custom mixer operates in software at 44100 Hz stereo signed 16-bit. It does not support hardware-accelerated mixing or sample rates other than 44100 Hz without pre-resampling. Audio playback latency is bounded by the SDL3 audio buffer size and is not independently measured.

No response-contingent triggers. Keypresses are logged but do not currently trigger DLP output lines, which some EEG/MEG acquisition protocols require.

Future planned developments include per-row trigger configuration in the CSV format, response-contingent trigger output, a formal timing validation protocol, and support for parallel-port triggers on Linux.

Open Practices Statement

The source code for Gostim2, along with pre-compiled binaries for Windows, macOS, and Linux and complete documentation, is freely available under the Apache License 2.0 at https://github.com/chrplr/gostim2. All example experiments included in the repository (stimuli, CSV schedules, configuration files) can be run immediately after installation to verify correct operation.

Declarations

Funding None

Conflicts of interest / Competing interests The authors declare no competing interests.

Ethics approval Not applicable. This manuscript describes a software tool and contains no human subjects data.

Consent to participate / Consent for publication Not applicable.

Availability of data and materials Not applicable.

Code availability The source code is hosted at https://github.com/chrplr/gostim2 under the Apache License 2.0. Pre-built binaries for all supported platforms are available at https://github.com/chrplr/gostim2/releases.

Author Contributions

Christophe Pallier: Conceptualization, Software (original conception, architecture, all design decisions, integration and testing), Writing — original draft (conceptualization and framing), Writing — review and editing.

Claude Sonnet 4.6 (Anthropic): Software (substantial contributions to engine modules including `engine/audio.go`, `engine/gui_setup.go`, `engine/run.go`, `engine/csv_parser.go`, `engine/validator.go`, and associated tests; co-development of the CLAUDE.md architectural context document), Writing — original draft (the revised paper text was drafted collaboratively in an interactive session).

Note on AI authorship: consistent with current editorial norms (Nature Portfolio, 2023; Springer Nature, 2023), Claude Sonnet 4.6 is listed under Author Contributions rather than as a named author, as authorship requires the capacity to assume accountability for the work. Christophe Pallier takes full responsibility for all content, including the AI-assisted portions.

References

Anwyl-Irvine, A. L., Massonnié, J., Flitton, A., Kirkham, N., & Evershed, J. K. (2021). Gorilla in our midst: An online behavioral experiment builder. *Behavior Research Methods*, *52*(1), 388–407. https://doi.org/10.3758/s13428-019-01237-x

Brainard, D. H. (1997). The Psychophysics Toolbox. *Spatial Vision*, *10*(4), 433–436. https://doi.org/10.1163/156856897X00357

Bridges, D., Pitiot, A., MacAskill, M. R., & Peirce, J. W. (2020). The timing mega-study: Comparing a range of experiment generators, both lab-based and online. *PeerJ*, *8*, e9414. https://doi.org/10.7717/peerj.9414

de Leeuw, J. R. (2015). jsPsych: A JavaScript library for creating behavioral experiments in a Web browser. *Behavior Research Methods*, *47*(1), 1–12. https://doi.org/10.3758/s13428-014-0458-y

Donovan, A. A. A., & Kernighan, B. W. (2016). *The Go programming language*. Addison-Wesley.

Kleiner, M., Brainard, D. H., & Pelli, D. (2007). What's new in Psychtoolbox-3? *Perception*, *36*(ECVP Abstract Supplement), 14.

Krause, F., & Lindemann, O. (2014). Expyriment: A Python library for cognitive and neuroscientific experiments. *Behavior Research Methods*, *46*(2), 416–428. https://doi.org/10.3758/s13428-013-0390-6

Nature Portfolio. (2023). *Tools and technologies: Artificial intelligence*. https://www.nature.com/nature-portfolio/editorial-policies/ai

Peirce, J. W., Gray, J. R., Simpson, S., MacAskill, M. R., Höchenberger, R., Sogo, H., Kastman, E., & Lindeløv, J. K. (2019). PsychoPy2: Experiments in behavior made easy. *Behavior Research Methods*, *51*(1), 195–203. https://doi.org/10.3758/s13428-018-01193-y

Schneider, W., Eschman, A., & Zuccolotto, A. (2002). *E-Prime user's guide*. Psychology Software Tools.

Springer Nature. (2023). *Artificial intelligence (AI) policy for authors*. https://www.springernature.com/gp/policies/ policies

Zyko0. (2024). *go-sdl3: Go bindings for SDL3*. GitHub. https://github.com/Zyko0/go-sdl3

OS Security Warnings for Unsigned Binaries

Gostim2 is distributed as a pre-compiled binary that is not signed with a commercial code-signing certificate. Both macOS and Windows apply security mechanisms that will warn users — or actively block execution — the first time an unsigned binary downloaded from the internet is run. This section describes the expected behavior and how to proceed.

macOS: GateKeeper

Apple's GateKeeper evaluates downloaded applications against three tiers of trust:

1. **Unsigned** — no signature at all. GateKeeper displays a dialog stating the application "is damaged and can't be opened." This phrasing is misleading: the binary is not corrupted; it simply lacks a signature.
2. **Ad-hoc signed** — signed with `codesign --sign -` at no cost, without an Apple Developer account. GateKeeper shows a one-time warning on first launch ("Apple cannot verify that this app is free of malware") but allows the user to proceed.
3. **Apple-notarized** — signed and notarized through Apple's notarization service, which requires a paid Apple Developer Program membership ($99/year). No warning is shown.

Gostim2 releases are distributed as ad-hoc signed `.app` bundles (macOS) or ad-hoc signed CLI binaries, so users will see a one-time first-launch warning rather than the alarming "damaged" message.

For graphical `.app` bundles: Right-click (or Control-click) the `.app` file and select "Open". A dialog will ask you to confirm; click "Open". This is required only once.

Alternatively: open System Settings → Privacy & Security. A message about the blocked app

appears at the bottom of the page with an "Open Anyway" button.

For command-line binaries: The quarantine extended attribute must be removed before first use:

```
xattr -d com.apple.quarantine gostim2
chmod +x gostim2
```

For detailed instructions and the packaging script used to create ad-hoc signed releases, see: https://chrplr.github.io/note-about-macos-unsigned-apps/

Windows: SmartScreen

Windows SmartScreen (part of Microsoft Defender) evaluates executables downloaded from the internet using a reputation system. An executable that has not accumulated sufficient download history — as is typical for software distributed outside the Microsoft Store or without an Extended Validation (EV) code-signing certificate — triggers a blue dialog: "Windows protected your PC. Microsoft Defender SmartScreen prevented an unrecognized app from starting."

To run Gostim2: click "More info" in that dialog, then click "Run anyway". The warning appears only once per executable file.

For the NSIS-based Windows installer (`gostim2-setup.exe`), the same dialog may appear before installation. The same "More info" → "Run anyway" procedure applies.

Obtaining an EV code-signing certificate from a commercial certificate authority would suppress this warning automatically but involves ongoing cost and is not currently pursued for this open-source project.

Note for Researchers Distributing Experiments to Participants

If you compile Gostim2 from source and distribute the resulting binary to participant computers, those computers will encounter the same warnings described above. The procedures are identical: on macOS, remove the quarantine attribute or use the right-click-Open method; on Windows, use "More info" → "Run anyway". Informing participants or technical staff in advance avoids confusion.